

# Reviewer Pack — Hilbert Polynomial Leading Coefficient Inversely Proportiona...

Entry f1e317557d3c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 06:36:45 UTC

## Hilbert Polynomial Leading Coefficient Inversely Proportional to Permanent Circuit Size

**Entry ID:** f1e317557d3c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 06:36:45 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Hilbert schemes and Hilbert polynomials)

**Field B** (complexity object): Boolean Circuit Size for Permanent Polynomial

#### Statement:

For an  $n \times n$  matrix  $M$ , the leading coefficient of the Hilbert polynomial of the ideal  $I$  defining the variety of matrices of rank  $\leq k$  is  $\Theta(1/\text{circuitsize}(n))$ , where  $\text{circuitsize}(n)$  is the minimal size of a circuit computing the permanent on  $n$  variables.

#### Rationale (proposer's reasoning):

The Hilbert polynomial encodes the structure of the ideal  $I$ , which captures the algebraic relations among the entries of the matrix. The leading coefficient relates to the dimension of the variety, which might reflect the complexity of the ideal's generators. Since the permanent's circuit size is a measure of its computational complexity, this conjecture links algebraic geometry to circuit complexity.

**Taxonomy category:** GCT (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** e159a9b675c0a9fd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        result = [[sum(A[i][k] * B[k][j] for k in range(p)) for j in range(n)] for i in range(m)]
        return result

    def determinant(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        det = 0
        for c in range(len(matrix)):
            submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
            sign = (-1) ** (c % 2)
            sub_det = determinant(submatrix)
            det += sign * matrix[0][c] * sub_det
        return det

    def permanent(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        perm = 0
        for c in range(len(matrix)):
            submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
            sign = (-1) ** (c % 2)
            sub_perm = permanent(submatrix)
            perm += sign * matrix[0][c] * sub_perm
        return perm

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
```

```

        max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
        A[i], A[max_row] = A[max_row], A[i]
        factor = 1 / A[i][i]
        for j in range(n):
            A[i][j] *= factor
        for k in range(n):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

def rank(matrix):
    rref = gaussian_elimination(matrix)
    rank = sum(1 for row in rref if any(row[j] != 0 for j in range(len(row))))
    return rank

def hilbert_polynomial(n, k):
    # This is a placeholder function. The actual implementation depends on the specific form of the polynomial.
    # For simplicity, we assume a known form here.
    return (n - k + 1) ** n / math.factorial(k)

def circuitsize(n):
    # Placeholder for the minimal circuit size for computing the permanent.
    # This is a simplified example. The actual implementation depends on the specific algorithm.
    return n * n

n = random.randint(5, 40)
k = random.randint(1, n-1)
M = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]

I = []
for r in range(k+1):
    for A in itertools.combinations(M, r):
        if rank(A) == r:
            I.append(A)

hp = hilbert_polynomial(n, k)
leading_coefficient = hp / math.factorial(k)
circuit_size = circuitsize(n)

conjecture_holds = abs(leading_coefficient - 1/circuit_size) < 1e-6
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Leading Coefficient",
    "metric_value": leading_coefficient,
    "instances_tested": len(I),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 35)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33c42b43.py", line 111, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33c42b43.py", line 87, in run_trial
    if rank(A) == r:
       ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33c42b43.py", line 66, in rank
    rref = gaussian_elimination(matrix)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33c42b43.py", line 54, in gaussian_elimination
    A[i], A[max_row] = A[max_row], A[i]
    ~~~
TypeError: 'tuple' object does not support item assignment

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed during Gaussian elimination due to tuple assignment error, preventing data collection | next: Debug and fix the Gaussian elimination implementation to handle matrix row operations properly

## 11. Audit log (LLM calls)

Total LLM calls: 7

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 79289        |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24759        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 21121        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 15419        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12877        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12240        |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 15971        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 181676 ms total latency. Provider mix: {'ollama\_remote': 7}

(full prompt+response transcripts available in `research/audit/f1e317557d3c.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f1e317557d3c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/f1e317557d3c.tar.gz` (if generated)